

```
import pandas as pd
```

```
from google.colab import files
```

```
uploaded = files.upload()
```

no files selected

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
df = pd.read_csv("student_performance_prediction.csv")
```

```
df.head()
```

	Student ID	Study Hours per Week	Attendance Rate	Previous Grades	Participation in Extracurricular Activities	Parent Education Level	Passed
0	S00001	12.5	NaN	75.0	Yes	Master	True
1	S00002	9.3	95.3	60.6	No	High School	True
2	S00003	13.2	NaN	64.0	No	Associate	True

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 40000 entries, 0 to 39999
```

```
Data columns (total 7 columns):
```

#	Column	Non-Null Count	Dtype
0	Student ID	40000 non-null	object
1	Study Hours per Week	38005 non-null	float64
2	Attendance Rate	38008 non-null	float64
3	Previous Grades	38006 non-null	float64
4	Participation in Extracurricular Activities	38000 non-null	object
5	Parent Education Level	38000 non-null	object
6	Passed	38000 non-null	object

```
dtypes: float64(3), object(4)
```

```
memory usage: 2.1+ MB
```

```
df.describe()
```

	Study Hours per Week	Attendance Rate	Previous Grades
count	38005.000000	38008.000000	38006.000000
mean	9.962744	75.276323	65.440107
std	5.031154	20.393418	16.503119
min	-12.300000	-14.300000	8.300000
25%	6.600000	61.600000	55.100000
50%	10.000000	75.300000	65.200000
75%	13.400000	88.800000	75.200000
max	32.400000	150.200000	200.000000

```
df.isnull().sum()
```

	0
Student ID	0
Study Hours per Week	1995
Attendance Rate	1992
Previous Grades	1994
Participation in Extracurricular Activities	2000
Parent Education Level	2000
Passed	2000

```
dtype: int64
```

```
df.shape
```

```
(40000, 7)
```

```
df.columns
```

```
Index(['Student ID', 'Study Hours per Week', 'Attendance Rate',  
      'Previous Grades', 'Participation in Extracurricular  
Activities',  
      'Parent Education Level', 'Passed'],  
      dtype='object')
```

```
df.isnull().sum()
```

	0
Student ID	0
Study Hours per Week	1995
Attendance Rate	1992
Previous Grades	1994
Participation in Extracurricular Activities	2000
Parent Education Level	2000
Passed	2000

dtype: int64

```
import matplotlib.pyplot as plt  
import seaborn as sns
```

```
df.head(10)
```

	Student ID	Study Hours per Week	Attendance Rate	Previous Grades	Participation in Extracurricular Activities	Parent Education Level	Passed
0	S00001	12.5	NaN	75.0	Yes	Master	True
1	S00002	9.3	95.3	60.6	No	High School	True
2	S00003	13.2	NaN	64.0	No	Associate	True
3	S00004	17.6	76.8	62.4	Yes	Bachelor	True
4	S00005	8.8	89.3	72.7	No	Master	True
5	S00006	8.8	73.8	69.3	Yes	High School	True
6	S00007	17.9	38.6	93.6	No	Doctorate	True
7	S00008	13.8	95.8	59.2	Yes	Doctorate	True

```
# Fill numerical missing values with the median
df["Study Hours per Week"] = df["Study Hours per Week"].fillna(df["Study Hours per Week"].median())
df["Attendance Rate"] = df["Attendance Rate"].fillna(df["Attendance Rate"].median())
df["Previous Grades"] = df["Previous Grades"].fillna(df["Previous Grades"].median())

# Fill categorical missing values with the mode
df["Participation in Extracurricular Activities"] = df["Participation in Extracurricular Activities"].fillna(df["Participation in Extracurricular Activities"].mode()[0])
df["Parent Education Level"] = df["Parent Education Level"].fillna(df["Parent Education Level"].mode()[0])
df["Passed"] = df["Passed"].fillna(df["Passed"].mode()[0])
```

Double-click (or enter) to edit

```
df.isnull().sum()
```

	0
Student ID	0
Study Hours per Week	0
Attendance Rate	0
Previous Grades	0
Participation in Extracurricular Activities	0
Parent Education Level	0
Passed	0

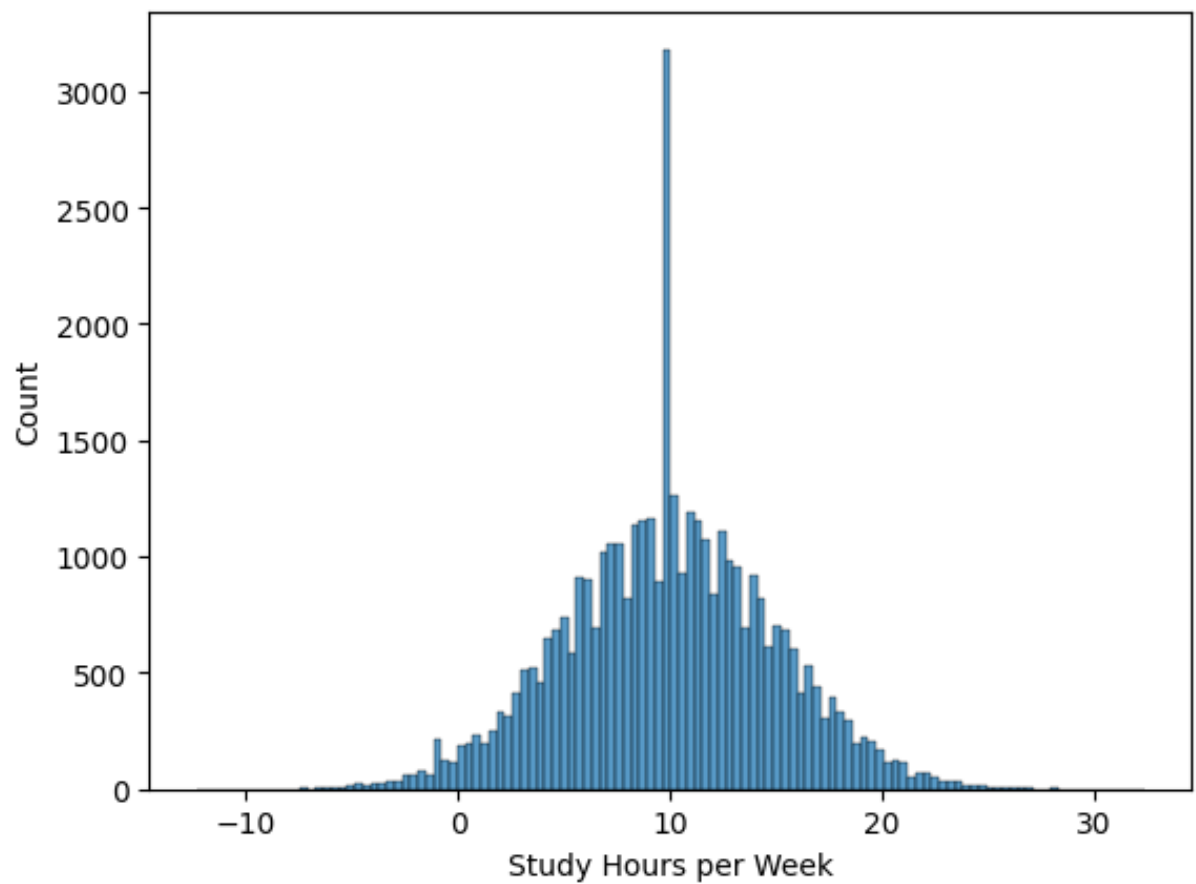
dtype: int64

Double-click (or enter) to edit

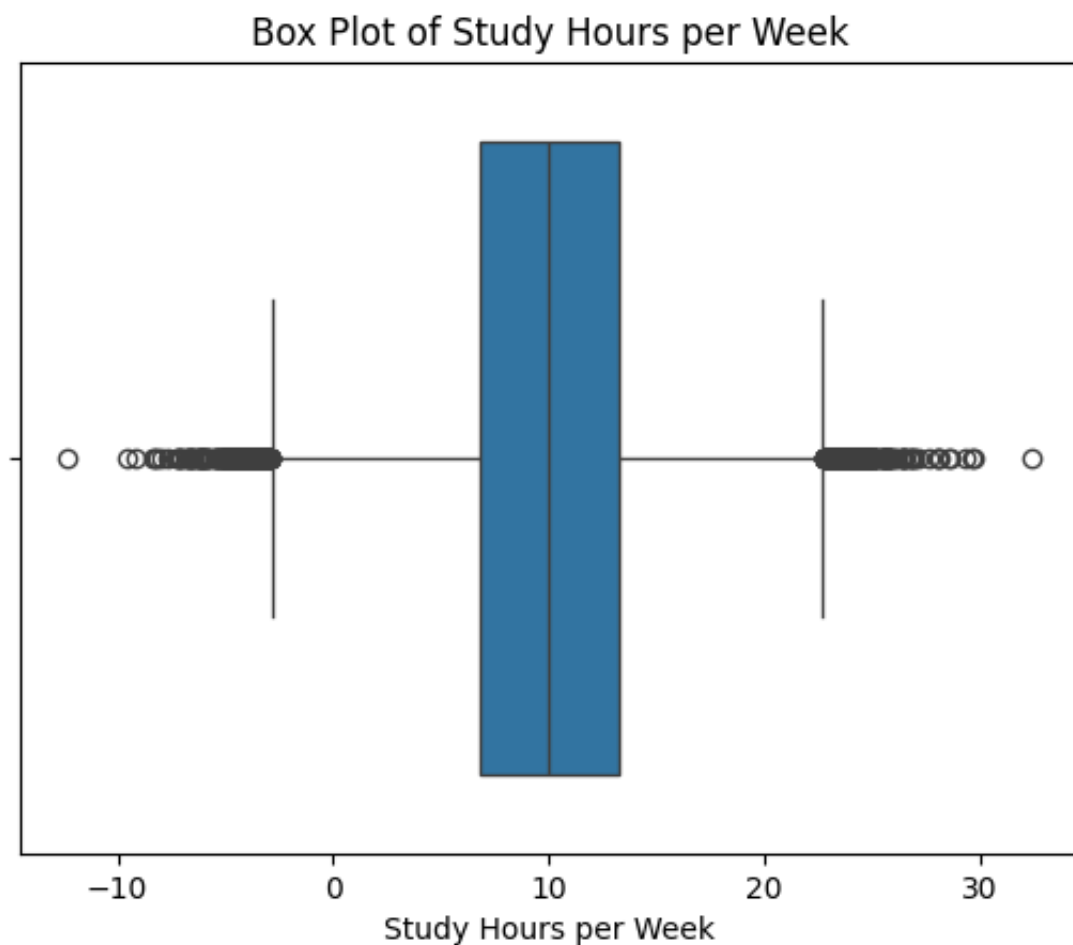
```
df.describe()
```

	Study Hours per Week	Attendance Rate	Previous Grades
count	40000.000000	40000.000000	40000.000000
mean	9.964602	75.277502	65.428138
std	4.904088	19.879125	16.086595
min	-12.300000	-14.300000	8.300000
25%	6.800000	62.400000	55.700000
50%	10.000000	75.300000	65.200000
75%	13.200000	87.900000	74.600000
max	32.400000	150.200000	200.000000

```
sns.histplot(df["Study Hours per Week"])  
plt.show()
```



```
sns.boxplot(x=df["Study Hours per Week"])
plt.title("Box Plot of Study Hours per Week")
plt.show()
```

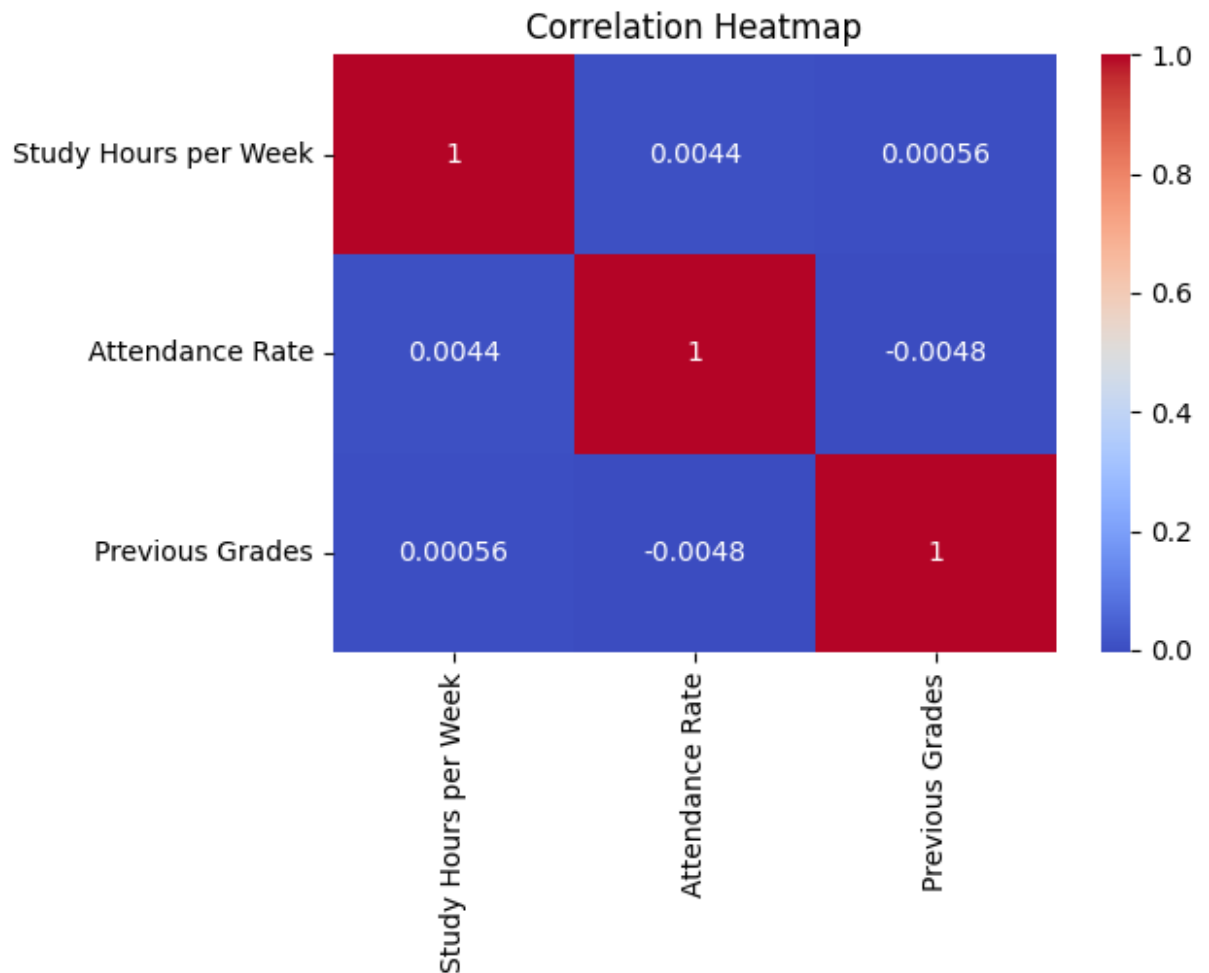


```
df.describe()
```

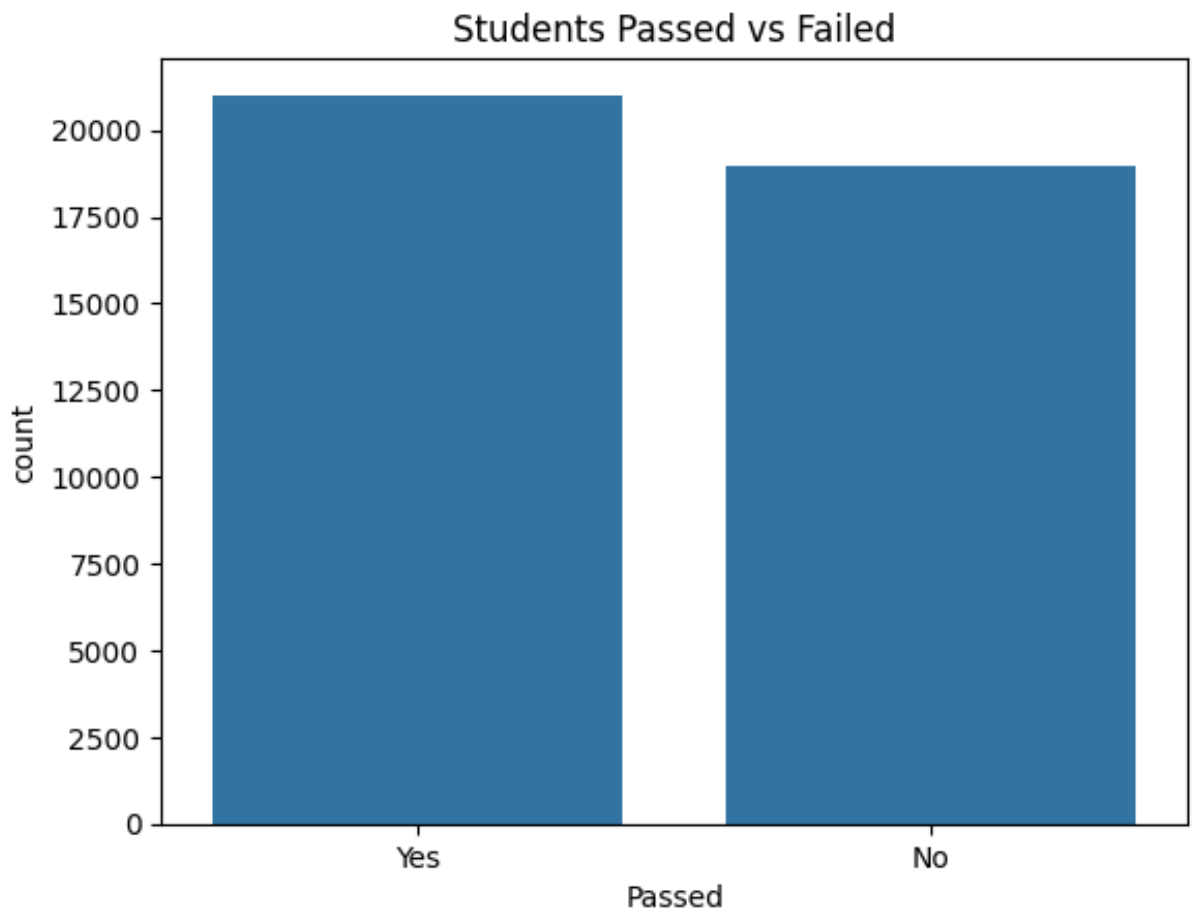
	Study Hours per Week	Attendance Rate	Previous Grades
count	40000.000000	40000.000000	40000.000000
mean	9.964602	75.277502	65.428138
std	4.904088	19.879125	16.086595
min	-12.300000	-14.300000	8.300000
25%	6.800000	62.400000	55.700000
50%	10.000000	75.300000	65.200000
75%	13.200000	87.900000	74.600000
max	32.400000	150.200000	200.000000

```
numeric_df = df.select_dtypes(include=["number"])

plt.figure(figsize=(6,4))
sns.heatmap(numeric_df.corr(), annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```




```
sns.countplot(x="Passed", data=df)
plt.title("Students Passed vs Failed")
plt.show()
```



```
from sklearn.preprocessing import LabelEncoder
```

```
le = LabelEncoder()
```

```
df["Participation in Extracurricular Activities"] = le.fit_transform(df["Participation in Extracurricular Activities"])
df["Parent Education Level"] = le.fit_transform(df["Parent Education Level"])
df["Passed"] = le.fit_transform(df["Passed"])
```

```
X = df.drop(["Student ID", "Passed"], axis=1)
y = df["Passed"]
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)
```

▼ LogisticRegression ⓘ ?

```
LogisticRegression(max_iter=1000)
```

```
y_pred = model.predict(X_test)
```

```
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, y_pred)

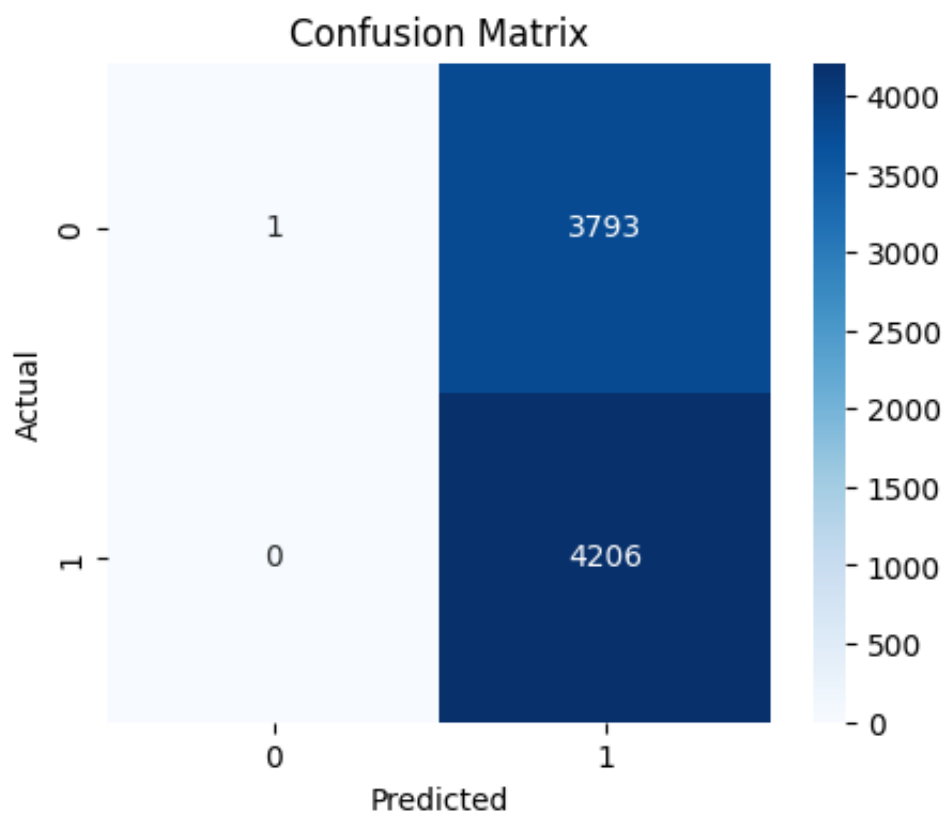
print("Accuracy:", accuracy)
```

Accuracy: 0.525875

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```



```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	1.00	0.00	0.00	3794
1	0.53	1.00	0.69	4206
accuracy			0.53	8000
macro avg	0.76	0.50	0.34	8000
weighted avg	0.75	0.53	0.36	8000

```
feature_importance = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": model.coef_[0]
})

print(feature_importance.sort_values(by="Coefficient", ascending=False))
```

	Feature	Coefficient
4	Parent Education Level	0.011663
3	Participation in Extracurricular Activities	0.003481
1	Attendance Rate	0.000628
2	Previous Grades	0.000042
0	Study Hours per Week	-0.004542

```
import joblib

joblib.dump(model, "student_performance_model.pkl")

print("Model saved successfully!")
```

Model saved successfully!

```
feature_importance = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": model.coef_[0]
})

feature_importance = feature_importance.sort_values(by="Coefficient")
print(feature_importance)
```

	Feature	Coefficient
4	Parent Education Level	0.011663
3	Participation in Extracurricular Activities	0.003481
1	Attendance Rate	0.000628
2	Previous Grades	0.000042
0	Study Hours per Week	-0.004542

```
new_student = pd.DataFrame({
    "Study Hours per Week": [25],
    "Attendance Rate": [90],
    "Previous Grades": [85],
    "Participation in Extracurricular Activities": [1],
    "Parent Education Level": [2]
})

prediction = model.predict(new_student)

print("Prediction:", "Pass" if prediction[0] == 1 else "Fail")
```

Prediction: Pass

```
results = X_test.copy()
results["Actual"] = y_test
results["Predicted"] = y_pred

results.to_csv("student_predictions.csv", index=False)

print("Predictions saved successfully!")
```

Predictions saved successfully!

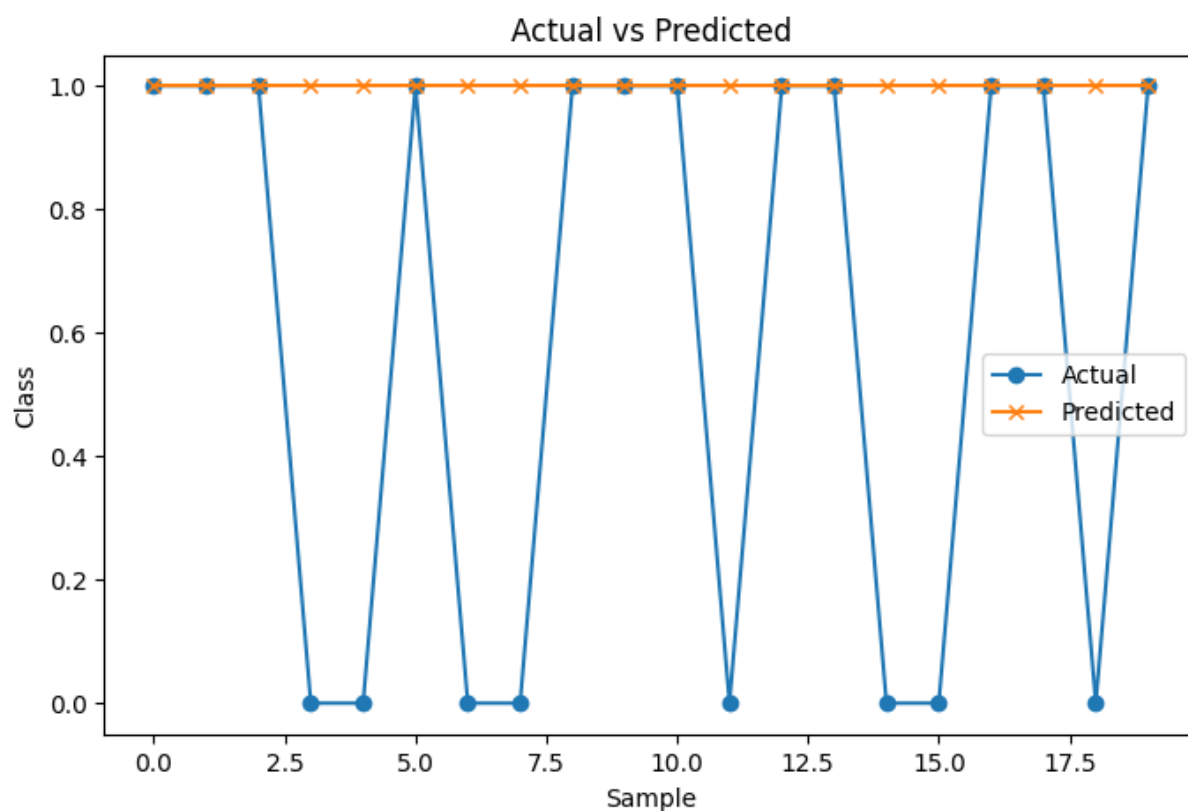
```
results = pd.DataFrame({
    "Actual": y_test.values,
    "Predicted": y_pred
})

print(results.head(10))
```

	Actual	Predicted
0	1	1
1	1	1
2	1	1
3	0	1
4	0	1
5	1	1
6	0	1
7	0	1
8	1	1
9	1	1

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))
plt.plot(results["Actual"].values[:20], marker='o', label='Actual')
plt.plot(results["Predicted"].values[:20], marker='x', label='Predicted')
plt.title("Actual vs Predicted")
plt.xlabel("Sample")
plt.ylabel("Class")
plt.legend()
plt.show()
```



```
results.to_csv("prediction_results.csv", index=False)
print("Prediction results saved successfully!")
```

Prediction results saved successfully!

```
print("===== MODEL SUMMARY =====")
print("Algorithm : Logistic Regression")
print("Training Samples :", len(X_train))
print("Testing Samples :", len(X_test))
print("Accuracy :", accuracy)
print("=====")
```

```
===== MODEL SUMMARY =====
Algorithm : Logistic Regression
Training Samples : 32000
Testing Samples : 8000
Accuracy : 0.525875
=====
```

```
from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt

y_prob = model.predict_proba(X_test)[:, 1]

fpr, tpr, thresholds = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(6,5))
plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.2f}")
plt.plot([0,1], [0,1], linestyle="--")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()
```

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(model, X, y, cv=5)

print("Cross Validation Scores:", scores)
print("Average Accuracy:", scores.mean())
```

```
Cross Validation Scores: [0.52525  0.525375 0.52525  0.525375 0.525
Average Accuracy: 0.52525
```



```
prediction_df = X_test.copy()
prediction_df["Actual"] = y_test
prediction_df["Predicted"] = y_pred

prediction_df.to_csv("Final_Predictions.csv", index=False)

print("Final predictions saved successfully!")
```

Final predictions saved successfully!

```
print("="*50)
print("STUDENT PERFORMANCE PREDICTION PROJECT")
print("Status : COMPLETED")
print("Algorithm : Logistic Regression")
print("EDA : Completed")
print("Model Training : Completed")
print("Evaluation : Completed")
print("="*50)
```

```
=====
STUDENT PERFORMANCE PREDICTION PROJECT
Status : COMPLETED
Algorithm : Logistic Regression
EDA : Completed
Model Training : Completed
Evaluation : Completed
=====
```

